Design Overview: 1
Requirements and Implementation: 1
System architecture and Modular design: 4
Why divide my program into modules? 5
Communication Between Modules 5
Overall layout: 5
Login System 6
Privilege hierarchy (most to least) 6
Flowchart: 7
Parking management 8
Accommodation simulation: 9
Parking Simulation 9
Accommodation management 9
Ski slope simulation: 9
Rental simulation: 9
Memberships and packages: 9
Database and data design: 9
Object-oriented design: 9
Algorithm design: 9
User interface design: 9
Validation, security and error handling: 9
Testing: 9

I HAVE DECIDED TO UTILISE A WEBSITE FOR MY DESIGN, SO A FEW THINGS HAVE BEEN DELETED/REWORKED TO HOUSE THIS

Design Overview:

My proposed solution is designed to effectively handle the flow of data within the ski resort: managing guests (handling bookings, appointments with certain services such as skiing, accommodation)

Requirements and Implementation:

Guest requirement -----> design implementation	
Requirements	Design implementation
Accommodation booking	Program prompts user for data into their stay; (dates, guests, room type) desired dates, and room preferences(used for the search algorithm); checks availability dynamically and confirms booking. Occupied rooms are automatically hidden by the system, unless user requests data (this seems like an administrative/staff level of action, so this may be reserved for the staff only)
Equipment renting	User selects equipment type and rental duration; system checks real-time availability, The system accesses 'equipment' class instances, such as ski poles, checking the availability attribute, or attribute that provides number of available units in integer form. calculates rental price based on duration and equipment type.
Membership management	Users can create, update, and view membership details (through use of getters and setters(methods) within the class); system applies tiered membership benefits and loyalty points automatically. user attributes are stored in the Customer class, including membership tier and activity logs. Activity logs stored in a stack

	data structure (list or stack class) to track recent actions, enabling undo or audit functionality. Membership benefits applied via polymorphism in membership subclasses (Silver, Gold, or Platinum).
Search & sort for accommodation	Users can filter by number of beds, price, distance from slopes, and amenities; sorting options include price, rating, and distance. Search filters utilise returned values from methods in the Accommodation class, filtering by attributes such as numbeds, price, and distance. Use of merge sort to organize available accomodation with respect to price, distance, or beds If user requests access to a certain room, isempty method is used to determine vacancy(each room will have a boolean attribute which determines vacancy or occpuancy of the room)
Package customisation	Customers can select and combine modular resort packages (e.g., all-inclusive, spa, food); system updates pricing and benefits accordingly. Use of classes; sustomers select modular packages represented as objects. Use of encapsulation to combine selected package modules into a single package object, stored as an attribute within the user's class profile

Staff requirement -----> design implementation	
Monitoring:	
Requirements	Design implementation
Membership management	Staff interface allows for the viewing and editing of member details, membership tiers, and activity logs. Activity logs retrieved from stack structure for recent member actions.

Accommodation metadata (time of booking, duration of stay, guest information)	Metadata stored as attributes in Reservation and Customer classes (z.B. booking_time, or stay_duration). The system logs and displays booking timestamps, stay duration, and guest profiles
Accommodation availability	Real-time dashboard on accommodation attributes, such as room occupancy, upcoming reservations, and vacancy forecasts. If user requests access to a certain room, isempty method is used to determine vacancy (each room will have a Boolean attribute which determines vacancy or occupancy of the room)
Slope traffic	Slope usage simulated and monitored through a SkiSlope class, containing attributes such as current_capacity and max_capacity. monitoring of slope usage (output attribute of the of the number of users on the slope), alerts staff when capacity limits are approached or exceeded.
reservations	Staff can view, modify, and cancel reservations; system tracks reservation status and notifies customers. Reservation status stored as an attribute (reserved, occupied, cancelled). Reserved status will automatically update occupied attribute to true

System requirements -----> design implementation	
Requirements	Design implementation
Slope simulation	Iterative program on all levels of slope (green blue, red, black) with imported 'random' module; use of classes to distinguish level of slope being affected by random iteration (higher level of slope, the lower chance of random occupancy)
OUTPUT Slope traffic	Output the population of the slope(s) , use of class attributes, population attribute which contains population (INT), and

	another which shows guests which are at the slope at any given time. alerts staff when capacity limits are approached or exceeded and blocks users and random iteration programme from adding to the queue. Use of the getter method isfull() and isempty()
Accommodation simulation (simulates traffic of ski resort accommodation)	Iterative program with imported 'random' module; on all levels of accommodations (three levels of accommodation, that increase in space, beds, view, therefore price) with imported 'random' module; use of classes to distinguish level of accommodation, being affected by the random iteration (higher level of accommodation, the lower chance of random occupancy). Occupied rooms are added to the cue, use of classes an attribute containing duration of stay determines when room exits the cue
Accommodation traffic (reservations and output of simulation and user occupancy)	Output the accommodations that have been reserved, use of queues, and removing rooms that are vacant from a queue containing occupied rooms.
Parking simulation	Iterative program with imported 'random' module. Occupied spaces are added to the cue, refers to the connecting attribute of duration of stay, from accommodation, to determine whether a parking spot exits the queue
Show Parking space traffic	Output the parking spaces that have been reserved (showing attributes such as duration of stay (foreign key from accommodation)), and the ones which are left, use of queues, removing rooms that are vacant from a queue containing occupied rooms
Data store of guest metadata; package, membership status, number of accompanying guests, rooms booked (this will have its own connected attributes	SQLite3

which will be affected after user input), parking spot, rented equipment	
Update room availability	Related to the simulation, accept it changes with user input

System architecture and Modular design:

Why divide my program into modules?

Dividing my ski resort management system into distinct modules improves organization, promoting clarity, subsequently easier troubleshooting. Each module should handle a unique aspect of the system, such as customer membership, equipment rental, accommodation booking, or slope management.

This separation ensures that each part can be developed, tested, and maintained independently without affecting or being affected by other modules; it will be much easier to discern any logic errors. It also aligns with OOP (Object-Oriented Programming) principles, where classes and objects encapsulate related data and behaviours.

Modular design reduces the complexity of the system via decomposition into manageable components, making it easier to understand and develop, and upgrade. this also helps determine the relations between each module, class, and variable within the program

Modular design also allows for me to easily discern which privileges and permissions i should give or revoke in terms of data access, depending on the role of the user (admin, guest, or staff member); The system differentiates access to data as such in order to ensures sensitive information is only shown to authorised users.

Communication Between Modules

Modules communicate will via shared data structures e.g.

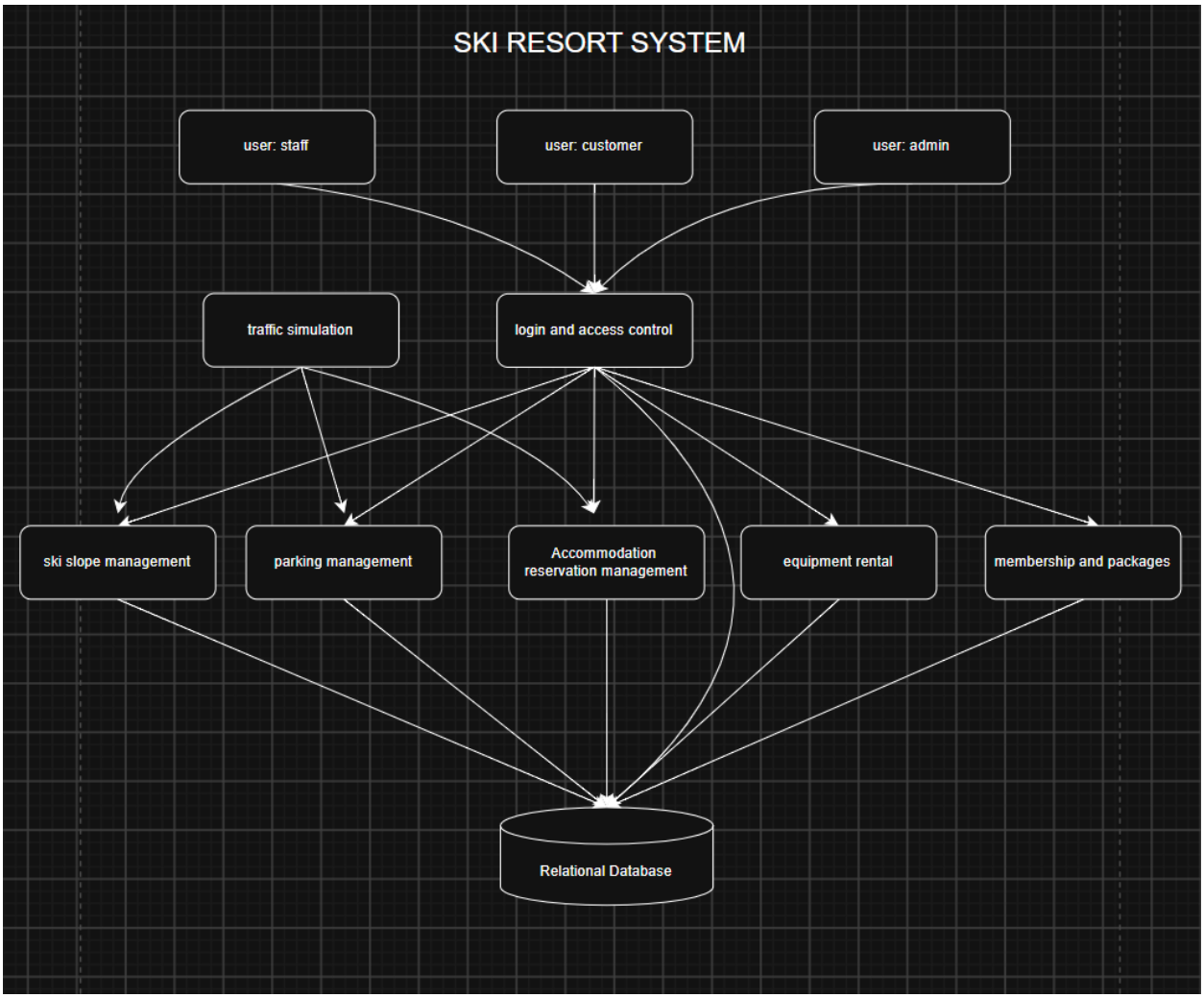
The Membership Module provides membership status and benefits information to the Booking Module to apply discounts or priority booking.

The Equipment Rental Module updates availability status, which the Reservation Module queries to prevent double bookings.

A relational database should act as an intermediary, allowing all modules to read and write data consistently using relational tables linked via foreign keys.

Communication is successful through method calls between class instances, 'event-driven updates' (notifying the parking module when a reservation is made, or an accommodation linked to the respected user is), or shared access to the database.

Overall layout:



Login System

all users connect to the same system, but they do not have the same privileges.

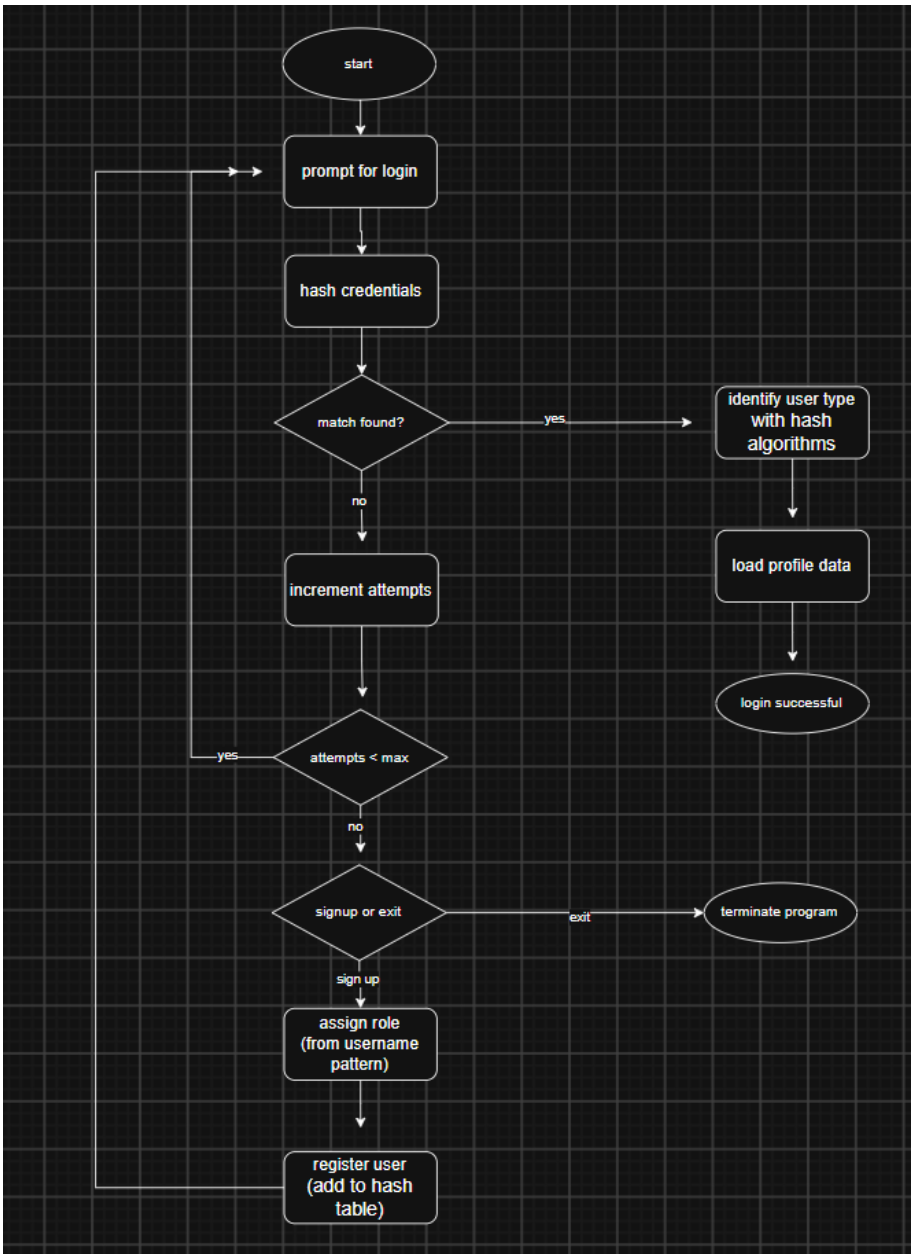
Privilege hierarchy (most to least)
1. **Administrators** have full access, including managing memberships, modifying system settings, and viewing detailed logs.
2. **Members of Staff** have broader access to monitor reservations, slope traffic, and equipment availability but may not see sensitive personal data unless necessary.
3. **Customers** can access their own membership details, bookings, equipment rentals, and package customizations.

The user will be prompted to enter their username and password. These credentials are processed through three distinct hashing algorithms, each corresponding to one of the three user types, customer, staff, admin. The hashed input will then be compared aga inst the respective hash tables to verify the login details. If a match is successful within the staff, customer or administrator hash table, the system determines the user's status based on their username and loads their profile and associated data and attributes into the program.

If the username or password is incorrect, a while loop will be triggered, repeatedly prompting the user to enter valid credentials. If the user provides the incorrect login details again, the program offers the option to sign up the user under a specific role, with the username format guiding the assignment of status (for example, staff usernames follow a special structure). If the user fails to provide valid credentials, upon promotion if the user accepts the proposal, the program terminates.

Flowchart:

The flowchart of this program(relational database is loaded with all names):



Profile maker:

the user profile curator is responsible for generating fake guests who can be used throughout the different simulations within my ski resort management system. The algorithm will utilise decomposition, dividing the process of generating a complete user

profile into several smaller subroutines, each responsible for generating specific attributes of the guest to provide the full profile. z.B, the generateName() function will randomly select a first name and surname from predefined sets, while a generateDOB() function will produce a random date of birth within a realistic age range. Another function would be generateStayDuration(), determining how long the guest intends to stay at the resort; for testing I can make this seconds, but due to the utilisation of a website in my solution being able to autonomously keep the date and therefore check if the stay duration should be terminated, i could make this generateStayDuration() generator generate a date of a week or more. Additional functions will assign random attributes relating to the guest's membership tier(affects when certain made up guests are allowed to be used by another simulation, as updated membership means earlier access times), package plan(affects what simulations can occupy certain guest data, as certain guests depending on plan can access certain things), rented equipment and accommodation preferences (their chosen level of accommodation. All of these attributes will be curated from pre-defined conditions, ensuring that the randomly generated profiles meet my own, provided standards, and remains compatible with the rest of the program (method is standardized).

once a profile has been generated, the system will provide the guest a 'guestID' before finally storing their information within the relational database; this field can then be used as a foreign key in other algorithms. An additional queue (activeguests[]) will be utilised by other programs, specifically the simulation programs, containing references to the profiles stored within the database rather than showing all of a profile's data (abstraction used for security for profiles that are not that of the administrator). The queue will use front and rear pointers to keep track of guests, with departing guests being logically removed from the active queue when their stay has ended so other simulations do not select a user that is not actively in teh hotel. However, the original guest records will remain stored within the relational database, allowing for their information to be accessed again if required, persay if the guest decided to stay at the resort again.

This approach is quite useful as multiple simulations will need access to the same guest profiles. z.B, the slope traffic simulation may randomly select an existing guest from the database and assign them to a particular slope, while the accommodation simulations simultaneously recognises that the same guest occupies a hotel room. additionally the equipment rental and parking simulations and management systems will be able to retrieve information from the guest's profile to determine whether they have rented equipment or have an associated parking space. By using a shared database approach, the system passively protects itself from creating/having multiple profiles appear in many simulations simultaneously, maintaining consistency between modules and allow

changes made by one simulation to be reflected across the rest of the program, further improving the simulations' realism

OR - **ask sir**

Approach where users have an 'activity' attribute, such that when a simulation tries to request a guest, unless the guest is already in another activity (lets say if activity = Ski or Spa) that simulation can then update this attribute, so no other simulations can access the user, until the activity is finished. This ensures that no one user is in two different simulations replicating activities at the resort (more realistic), and makes programming for this section easier as no front/rear pointer is required, and guests are only popped when their stay is terminated

Accommodation simulation:

The accommodation simulation will consist of its main program and two subroutines, one subroutine that removes occupancy attribute from a room , and one that adds occupancy, also attaching random credentials to the room,

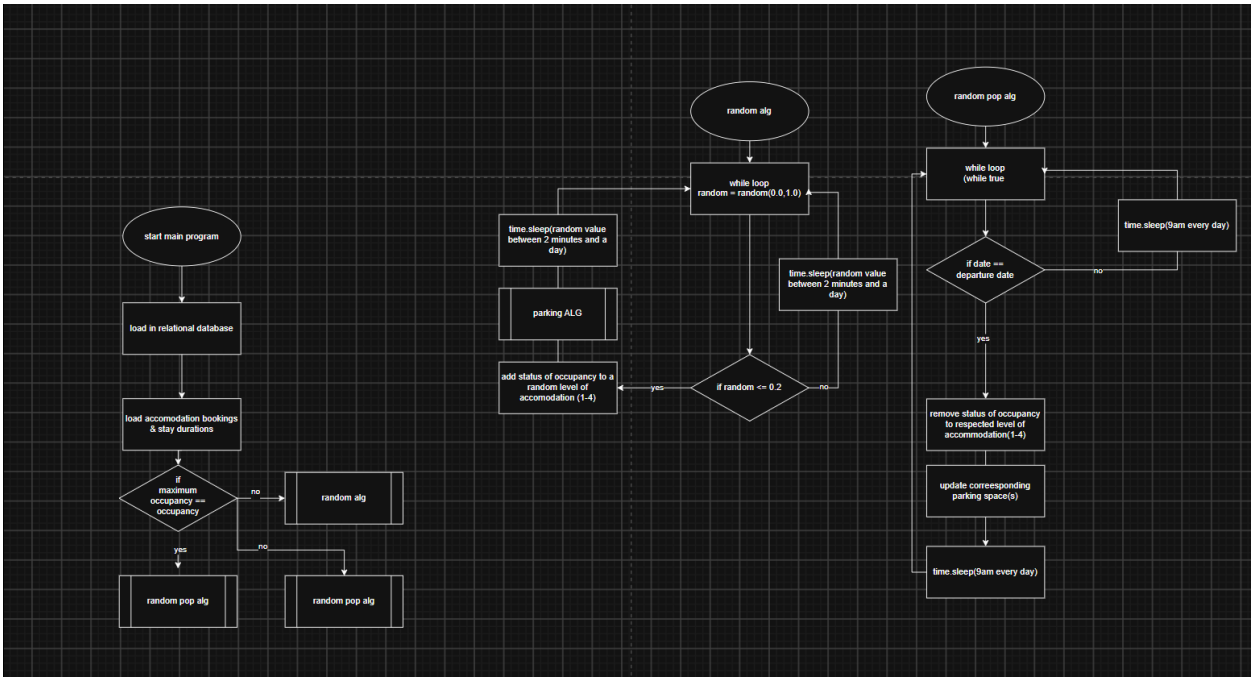
The idea is that all simulations should run in the background, updating occupancies and vacancies autonomously once started.

For testing, the random value for the time taken for the while loop to repeat will be reduced to a range of a few seconds

This directly influences the parking simulation, which will be shown in more detail in its respected section

Make the system even more realistic by making higher tier/better accommodations less likely to be booked; more expensive options are less likely to be booked than more common options

Flowchart:



Parking Simulation:

The accommodation simulation affects this simulation, and the two interact via the relational database. Instead of being a seperate generator of random parking occupancy, having no relationship to the rest of the program or the guests (degrading realism), the parking algorithm will execute after the accommodation simulation whenever a new guest checks into a room, and allocate the parking space(s) for that guest. When this happens, the accommodation simulation will log the occupancy event along with the unique identification number of the guest, the reservation ID, the room number, and the expected checkout date. The parking algorithm will then use the most recent unprocessed occupancy event to find which guest the algorithm should operate on. After the guest's details have been fetched, the algorithm will examine whether the reservation has any parking spaces allocated to it yet to avoid erroneously allocating multiple spaces to the same reservation. If no parking spaces have been allocated to the reservation, the algorithm will use 'Random', specifically random.randint(0,2) to randomly decide if the guest will occupy 0, 1, or 2 parking spaces (the guest in reality might travel by coach or private hire, which is why they may also not occupy any spots). Prior to the allocation of these parking spaces, the program will examine the availability of parking using the appropriate attributes stored within the database. If enough parking spaces are available(using a simple is_occupied() method to fetch the boolean value of the occupancy of the parking space, the algorithm will randomly choose the necessary number of empty parking spots to be assigned to the reservation and mark their occupancy status as

occupied. If only one parking space remains empty and two were allocated, the algorithm will assign the last space instead of going over the maximum parking spaces allowed. The allocated parking spaces will be associated with the guest's reservation via the use of primary and foreign keys within the relational database. This will enable the parking simulation to obtain details about the guest's lodging, such as their expected checkout date, without needing to store the information twice. The algorithm will also mark that the accommodation occupancy event has been processed so that the same guest will not have additional parking spaces assigned to them during the next iteration. The parking simulation will also have a seperate procedure dedicated to the removal of parking occupancy. This procedure will examine if the guest has reached their checkout date or has checked out of their accommodation. If the guest's stay has ended, all parking spaces associated with their reservation will have their occupancy status changed to empty, therefore becoming unoccupied. This establishes a direct link between occupancy of accommodation and parking to ensure that both simulations are in sync with each other.

Accommodation management:

New solution using SQLite3;

Instead of using a queue and logically popping accomodations, therefore only showing vacant rooms for posible room transfer

Parking management:

parking management system allows users to view and manage parking spaces assigned to their accommodation reservation/booking; as described before, by putting all parking spaces with their 'occupied' attribute being false in a queue (NOT EFFICIENT, USE SQL LITE3). Once a user (real user) has made a booking, the system will check whether they require parking and, if so, allocate either one or two parking spaces depending on their requirements. The allocated parking space IDs will be stored in an array attached to the user's booking details and displayed as an attribute in their user profile. This will only be viewable by the respective user, authorised staff of the hotel and administrators, to ensure that no unauthorised person has access to sensitive data.

[INSERT FLOWCHART LOADING SQLITE3 AND AND ACCOM MANAGEMENT INPUT AND OUTPUT FOR THIS PROCESS ONCE FOUND]

The parking area will be split into five different sectors: Red, Blue, Green, Yellow and Purple (for the sole reason of simplictiy, each sector will have 20 parking spaces, but this could be scaled up later). There will be an ordered set of parking spaces within each sector, holding a set of integer IDs, with a Boolean is_occupied attribute, which denotes whether the

space is available or not. If the user requires two parking spaces, the program will perfom a search algorithm which will iterate through each of the parking sectors, looking for two consecutive parking spaces that are available.

[INSERT THE LEVEL 1 DIAGRAM OF SEARCH ALGORIGHTM AND USER INPUT FOR NUMBER OF SPOTS REQUIRED]

This will involve comparing the adjacent parking space IDs whilst also checking whether both spaces have an occupancy value of False. For instance, if parking spaces 12 and 13 within the Red sector both have their occupancy value as False, the algorithm will identify that as a valid pair. When a valid pair has been found, both parking spaces will be allocated to the user, and their occupancy value will change. If there are no consecutive parking spaces that are free within any of the sectors, the program will provide the use

2. Random Parking Space Allocation Algorithm

For guests who only need one parking space, the random allocation algorithm will be used to determine which space the user will receive. Firstly, a random colour will be chosen from a list that contains all five parking sectors using the random.choice() function in Python. Then, the program will obtain the list of parking spaces within that sector, and find which ones have an is_occupied value of False. A random integer ID will then be selected from the list of available parking spaces, and the corresponding parking space will be allocated to the user's booking details. If the randomly chosen sector does not have any free spaces, the program will attempt to select another sector that has available capacity

Instead of physically removing the allocated parking space from the list using the pop() method, the is_occupied value will be changed from False to True. Effectively removing the space from the list of available parking spaces, whilst keeping its original index and integer ID. This is especially important for the algortihm that searches for two consecutive spots.

When the user's accommodation ends, the parking management system will obtain all parking spaces allocated to the user's booking, and change their occupancy values back to False.

3. Alternative Design – Parking Attributes Stored Within Accommodation

An alternative design would be to have the allocated parking space IDs held directly within the accommodation or room object, rather than within the user's profile. With this approach, each accommodation object will have an additional attribute, for example, parking_spaces, that will hold an array of allocated parking space IDs. When a guest occupies accommodation, the parking allocation algorithm will update this attribute so that the parking simulation can obtain the guest's allocated parking spaces through their

accommodation details. When the guest's stay ends, the corresponding parking spaces will be freed up, and the accommodation's parking attribute will be cleared.

UPDATE-To ensure data consistency between parking management, accommodation management and user profile modules, the relational database will be the central location of all parking allocations. Each parking space will have a unique identification number, sector and occupancy status. There will be a separate parking allocation table which will contain the parking space ID and the reservation ID as foreign keys, so that a reservation that requires two parking spaces can have two parking records without duplicating the guest's personal details.

When parking spaces are allocated or freed, the appropriate database records will be updated and the changes will be reflected within the user's profile and parking simulation.

Ski slope simulation:

To make this as realistic as possible, I've decided to replicate real world systems by researching average opening times which I have found to be an average of around 8:30, and peak hours for ski slopes, reflecting increased traffic with a lower time interval between every loop for the occupancy

Rental simulation:

Memberships and packages:

Object-oriented design:

Define the classes, attributes, methods and relationships for relevant programs

Discusses all the classes within each module, the getters and setters (methods)

And the attributes

User interface design:

How the user views the encapsulated data; talk about the importance of encapsulation into improving clarity on the client side, increase in data security. Implement website into this design

For the user interface, I intend to implement a website into my solution. I believe that this is the best approach, as it allows autonomy for

Validation, security and error handling:

Validation algorithms

Testing:

Make sure to relate to the analysis where necessary; where i can improve on something said in the analysis

To add realism, simulations such as accommodation could reflect real life regarding the chances that a premium suite is booked, compared to one of the more standard suites; more expensive suites could have less of a chance at being randomly booked than a standard level one suite

Could possibly have a queue which stores non occupied parking spaces, and is added or popped from so available parking spaces can easily be searched

ADA promotes the use of more flowcharts, and an UML diagram

More flowcharts are necessary to increase marks

Instead of using queues for the sort system, showing available parking spaces depending on the boolean value of the occupied attribute, utilise sql lite3

UTILISE SQL LITE TO STORE GUESTS AFTER THEYVE BEEN PHYSICALLY (NOT LOGICALLY) POPPED FROM ACTIVE GUEST QUEUE

PARKING MANAGEMENT ADDITION: POSSIBLY LET USER INPUT THEIR NUMBER PLATE, HOWEVER THIS ADDS A FEW PROBLEMS- I WOULD HAVE TO MAKE AN ALGORITHM THAT PRODUCES BRITISH NUMEBR PLATES(reg ex) FOR THE OTHER SIMULATED GUESTS;FOR NOW IGNORE THIS AND FOCUS ON NEW SOLUTION FOR THE WEBSITE

SORT OUT FLOWCHARTS- FIND OLD STORE OF FLOWCHARTS ON DRAW.IO, ASK SIR IF THEYRE PERMA LOST

THEN REDO OOP DESIGN ON THE UPDATED DESCRIPTIONS